CCCC Software Metrics Report generated Sun Dec 1 15:53:59 2024	
Project Summary	Summary table of high level measures summed over all files processed in the current run.
Procedural Metrics Summary	Table of procedural measures (i.e. lines of code, lines of comment, McCabe's cyclomatic complexity summed over each module.
Object Oriented Design	Table of four of the 6 metrics proposed by Chidamber and Kemerer in their various papers on 'a metrics suite for object oriented design'.
Structural Metrics Summary	Structural metrics based on the relationships of each module with others. Includes fan-out (i.e. number of other modules the current module uses), fan-in (number of other modules which use the current module), and the Information Flow measure suggested by Henry and Kafura, which combines these to give a measure of coupling for the module.
Other Extents	Lexical counts for parts of submitted source files which the analyser was unable to assign to a module. Each record in this table relates to either a part of the code which triggered a parse failure, or to the residual lexical counts relating to parts of a file not associated with a specific module.
About CCCC	A description of the CCCC program.

Project Summary

This table shows measures over the project as a whole.

- NOM = Number of modules
  Number of non-trivial modules identified by the analyser. Non-trivial modules include all classes, and any other module for which member functions are identified.
- LOC = Lines of Code
  Number of non-blank, non-comment lines of source code counted by the analyser.
- COM = Lines of Comments
  Number of lines of comment identified by the analyser
- MVG = McCabe's Cyclomatic Complexity
  A measure of the decision complexity of the functions which make up the program.The strict definition of this measure is that it is the number of linearly independent routes through a directed acyclic graph which maps the flow of control of a subprogram. The analyser counts this by recording the number of distinct decision outcomes contained within each function, which yields a good approximation to the formally defined version of the measure.
- L_C = Lines of code per line of comment
  Indicates density of comments with respect to textual size of program

- M_C = Cyclomatic Complexity per line of comment
  Indicates density of comments with respect to logical complexity of program
- IF4 = Information Flow measure
  Measure of information flow between modules suggested by Henry and Kafura. The analyser makes an approximate count of this by counting inter-module couplings identified in the module interfaces.

Two variants on the information flow measure IF4 are also presented, one (IF4v) calculated using only relationships in the visible part of the module interface, and the other (IF4c) calculated using only those relationships which imply that changes to the client must be recompiled of the supplier's definition changes.

Metric	Tag	Overall	Per Module
Number of modules	NOM	4	
Lines of Code	LOC	32	8.000
McCabe's Cyclomatic Number	MVG	6	1.500
Lines of Comment	COM	29	7.250
LOC/COM	L_C	1.103	
MVG/COM	M_C	0.207	
Information Flow measure (inclusive)	IF4	1	0.250
Information Flow measure (visible)	IF4v	1	0.250
Information Flow measure (concrete)	IF4c	1	0.250
Lines of Code rejected by parser	REJ	0	

Procedural Metrics Summary

For descriptions of each of these metrics see the information preceding the project summary table. The label cell for each row in this table provides a link to the functions table in the detailed report for the module in question

Module Name	LOC	MVG	COM	L_C	M_C
IWorker	12	0	10	------	------
Worker	13	1	12	------	------
anonymous	7	5	7	------	0.714
string	0	0	0	------	------

Object Oriented Design

- WMC = Weighted methods per class
  The sum of a weighting function over the functions of the module. Two different weighting functions are applied: WMC1 uses the nominal weight of 1 for each function, and hence measures the number of functions, WMCv uses a weighting function which is 1 for functions accessible to other modules, 0 for private functions.
- DIT = Depth of inheritance tree
  The length of the longest path of inheritance ending at the current module. The deeper the inheritance tree for a module, the harder it may be to predict

its behaviour. On the other hand, increasing depth gives the potential of greater reuse by the current module of behaviour defined for ancestor classes.
- NOC = Number of children
  The number of modules which inherit directly from the current module. Moderate values of this measure indicate scope for reuse, however high values may indicate an inappropriate abstraction in the design.
- CBO = Coupling between objects
  The number of other modules which are coupled to the current module either as a client or a supplier. Excessive coupling indicates weakness of module encapsulation and may inhibit reuse.

The label cell for each row in this table provides a link to the module summary table in the detailed report for the module in question

Module Name	WMC1	WMCv	DIT	NOC	CBO
IWorker	8	8	0	1	2
Worker	1	1	1	0	2
anonymous	7	7	0	0	0
string	0	0	0	0	2

Structural Metrics Summary

- FI = Fan-in
  The number of other modules which pass information into the current module.
- FO = Fan-out
  The number of other modules into which the current module passes information
- IF4 = Information Flow measure
  A composite measure of structural complexity, calculated as the square of the product of the fan-in and fan-out of a single module. Proposed by Henry and Kafura.

Note that the fan-in and fan-out are calculated by examining the interface of each module. As noted above, three variants of each each of these measures are presented: a count restricted to the part of the interface which is externally visible, a count which only includes relationships which imply the client module needs to be recompiled if the supplier's implementation changes, and an inclusive count The label cell for each row in this table provides a link to the relationships table in the detailed report for the module in question

Module Name	Fan-out			Fan-in			IF4		
	vis	con	inc	vis	con	incl	vis	con	inc
IWorker	1	1	1	1	1	1	1	1	1
Worker	0	0	0	2	2	2	0	0	0
anonymous	0	0	0	0	0	0	0	0	0
string	2	2	2	0	0	0	0	0	0

Other Extents

Location	Text	LOC	COM	MVG

About CCCC

This report was generated by the program CCCC, which is FREELY REDISTRIBUTABLE but carries NO WARRANTY.

CCCC was developed by Tim Littlefair. as part of a PhD research project. This project is now completed and descriptions of the findings can be accessed at http://www.chs.ecu.edu.au/~tlittlef.

User support for CCCC can be obtained by mailing the list cccc-users@lists.sourceforge.net.

Please also visit the CCCC development website at http://cccc.sourceforge.net.